

Technical Report – NHS Analysis

Introduction

With the population rising, and dealing with the aftereffects of the COVID-19 pandemic, the resources of the NHS have to be assessed. The following review will analyse two main questions posed by the NHS:

1. Has there been adequate staff and capacity in the networks?
2. What was the actual utilisation of resources?

To complete the analysis, more analytical questions are asked of the data:

- How well utilised are staff compared to each month?
- Is there any link between missed appointments and the time between a booking an appointment and the appointment taking place?

The insights gained from this analysis will help determine whether the current capacity of the NHS is adequate or it needs to be increased.

Data Ingestion and Wrangling

The relevant libraries and the data sets were read (see figure 1).

```
# Import the necessary Libraries.  
import pandas as pd  
import numpy as np  
  
# Optional - Ignore warnings.  
import warnings  
warnings.filterwarnings('ignore')
```

Figure 1 - Python code showing importing Pandas and Numpy libraries.

The number of different sub-ICBs and ICBs within the datasets was identified. This was done by counting the individual sub-ICB names and ICB ONS codes. ONS codes were used because ICB names were not present within the dataset, allowing a distinct count of unique ICB locations. There are 106 different sub-ICBs and 42 ICBs (see figure 2).

```
# Determine the number of Locations.  
print(nc['sub_icb_location_name'].value_counts().count())  
print(ad['sub_icb_location_ons_code'].value_counts().count())  
print("There are",  
      nc['sub_icb_location_name'].value_counts().count(),  
      "different sub-icb locations.")  
  
106  
106  
There are 106 different sub-icb locations.  
  
# Determine the number of wider Locations.  
print(nc['icb_ons_code'].value_counts().count())  
print(f"There are {nc['icb_ons_code'].value_counts().count()} wider locations, however we do not have their names, only their ONS location codes.")  
  
42  
There are 42 wider locations, however we do not have their names, only their ONS location codes.
```

Figure 2 - Python code for the number of locations

The five sub-ICB locations with the highest appointment volumes were all located in southern England. This was achieved by creating a subset of the dataset which highlighted how many times each value appeared in the data sets (see figure 3).

```
# Determine the top five locations based on record count.
nc_record_count = nc['sub_icb_location_name'].value_counts()

print(f""The top 5 locations based on record count:
      {nc_record_count.head(5).to_string(header=False)}""")
```

The top 5 locations based on record count:

NHS North West London ICB - W2U3Z	13007
NHS Kent and Medway ICB - 91Q	12637
NHS Devon ICB - 15N	12526
NHS Hampshire and Isle Of Wight ICB - D9Y0V	12171
NHS North East London ICB - A3A8R	11837

Figure 3 – Python Code to determine the top five locations

An understanding of the different values for service setting, context type, national category, and appointment status was achieved. There are 5 different service settings, 3 different context types, 18 different national categories and 3 different appointment statuses. One of the appointment statuses was DNA, this is something to look at in more detail later on (see figure 4).

```
# Determine the number of service settings.
print(nc['service_setting'].value_counts())
print(f""There are {nc['service_setting'].value_counts().count()} different service settings.""")
```

```
service_setting
General Practice      359274
Primary Care Network  183790
Other                 138789
Extended Access Provision  108122
Unmapped             27419
Name: count, dtype: int64
There are 5 different service settings.
```

```
# Determine the number of context types.
print(nc['context_type'].value_counts())
print(f""There are {nc['context_type'].value_counts().count()} different context types.""")
```

```
context_type
Care Related Encounter  700481
Inconsistent Mapping   89494
Unmapped              27419
Name: count, dtype: int64
There are 3 different context types.
```

Figure 4 – Some examples of Python code to understand different aspects of the data sets.

The start and end dates for each data set were identified using min and max. It should be acknowledged that the appointment_regional dataset did not have an exact appointment date column, but it did have an appointment month column. A Series was added to the DataFrame by using the datetime method on the appointment month. The datetime method was applied across all of the data sets, to enable a suitable time series analysis (see figure 5).

```
# Determine the minimum and maximum dates in the nc DataFrame.
# Use appropriate docstrings.
print(nc['appointment_date'].max().date())
print(nc['appointment_date'].min().date())
print(f"""The National Category data set starts at {nc['appointment_date'].min().date()}, and ends at {nc['appointment_date'].max().date()}.""")

2022-06-30
2021-08-01
The National Category data set starts at 2021-08-01, and ends at 2022-06-30.
```

Figure 5 – An example of some Python code to check the start and end dates of the data sets.

Subset DataFrames were created for each of the top 5 sub-ICBs focusing on service setting, which were then filtered (see figure 6) to look at specific dates. This revealed that for each of the top 5 sub-ICBs, General Practice was the most popular.

```
# Filter the dates of the appointments.
nc_NorthWestLondon = nc_NorthWestLondon[
    (nc_NorthWestLondon['appointment_date'] >= '2022-01-01')
    &
    (nc_NorthWestLondon['appointment_date'] <= '2022-06-01')]

# Sense check the filtered DataFrame.
print(nc_NorthWestLondon.shape)
nc_NorthWestLondon
```

Figure 6 – Showing some Python code that filters the DataFrame.

The year and month were grouped and the busiest month was discovered by looking at the count of appointments total. The busiest month was November 2021 (see figure 7). The NHS Digital website (2021) says that there were 34.6 million appointments in November 2021.

```
# Determine the total number of appointments per month grouped by year and month.
nc['count_of_appointments'].groupby([
    nc['appointment_date'].dt.year,
    nc['appointment_date'].dt.month]
).sum().sort_values(ascending=False)
```

appointment_date	appointment_date	
2021	11	30405070
	10	30303834
2022	3	29595038
2021	9	28522501
2022	5	27495508
	6	25828078
	1	25635474
	2	25355260
2021	12	25140776
2022	4	23913060
2021	8	23852171

Figure 7 – Python code using the groupby method to see the busiest month.

Visualisations

Prepared the Python workstation for visualisations. To aid with the analysis, some data types were changed to a Python string (see figure 8).

```
# Import the necessary Libraries.
import seaborn as sns
import matplotlib.pyplot as plt

# Set figure size.
sns.set(rc={'figure.figsize':(15, 12)})

# Set the plot style as white.
sns.set_style('white')
```

Figure 8 – Python code preparing the workstation for visualisations.

Subsets of the data were created (see figure 9), and count of appointments were totalled so that a time series could be undertaken on the appointment month (see figure 10 and 11). There are far more counts of general practice compared to other settings.

```
# Aggregate on monthly level and determine the sum of records per month.  
nc_ss = nc.groupby(['appointment_month', 'service_setting'])\  
        ['count_of_appointments'].sum().reset_index()  
  
# View output.  
nc_ss.head()
```

Figure 9 – Python code showing subsets using the groupby method.

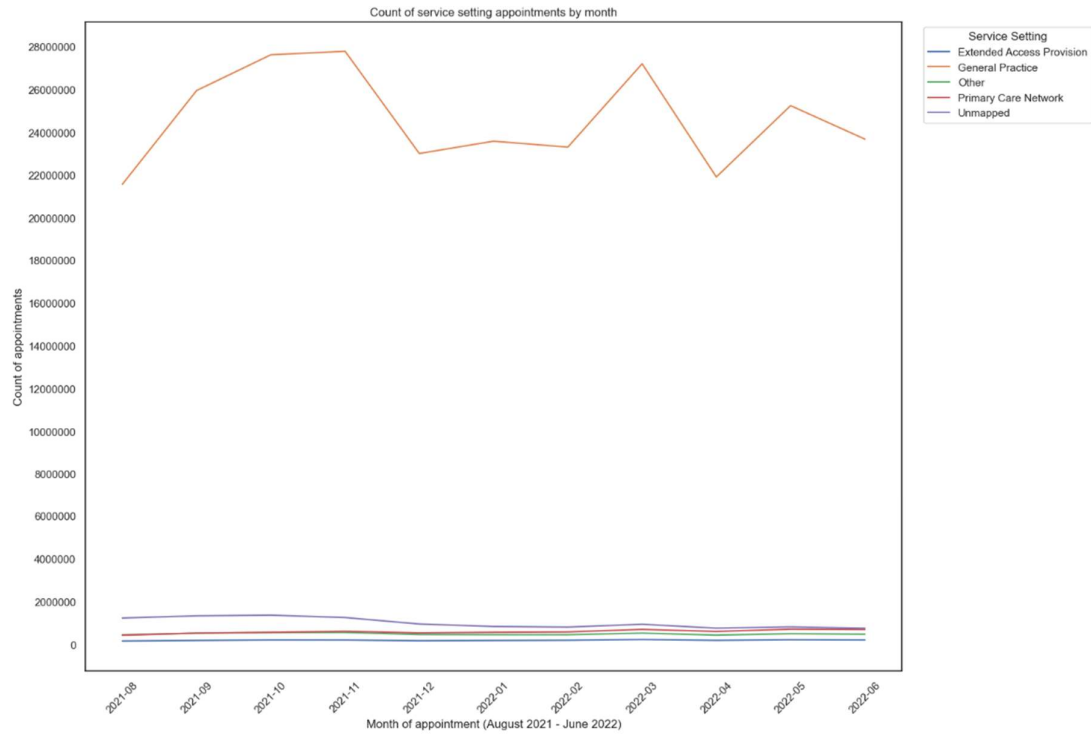


Figure 10 – Time series of service setting showing more General practice.

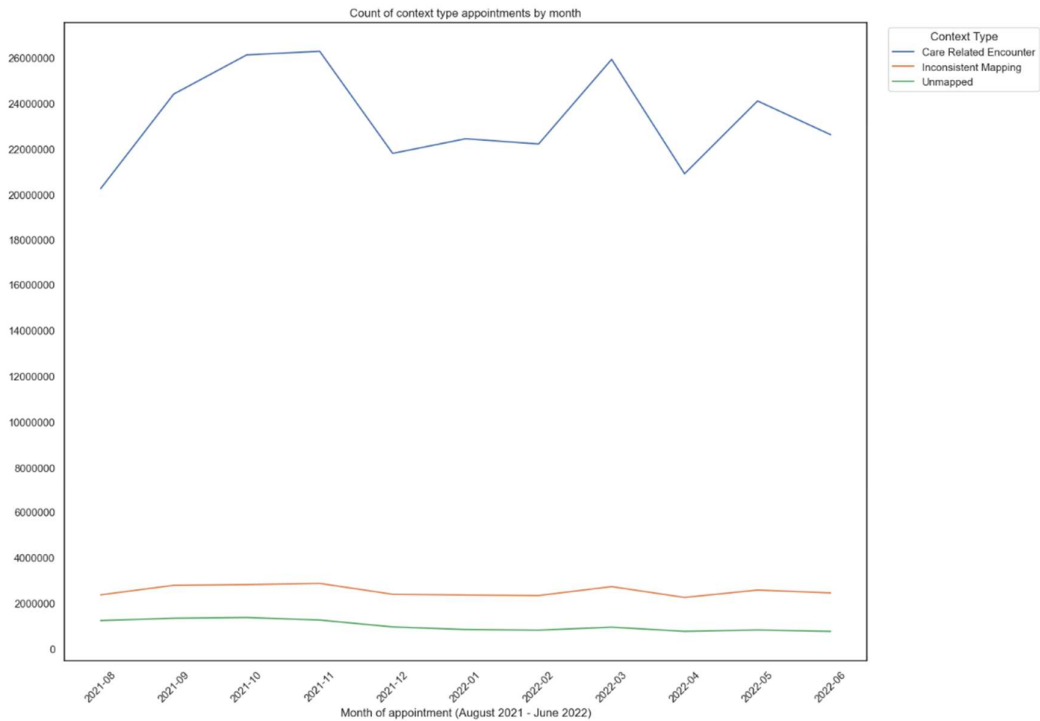


Figure 11 – a time series to show different context types over a year. Care related encounter is the most observed

Lineplots were created using a DataFrame of context type by season (see figure 12). The time series was taking appointment data into account. There are more appointments at the start of the week compared to the end. This is then followed by no appointments during the weekend. Perhaps if more capacity were required, appointments at weekends could be something to pursue, along with bank holidays as well.

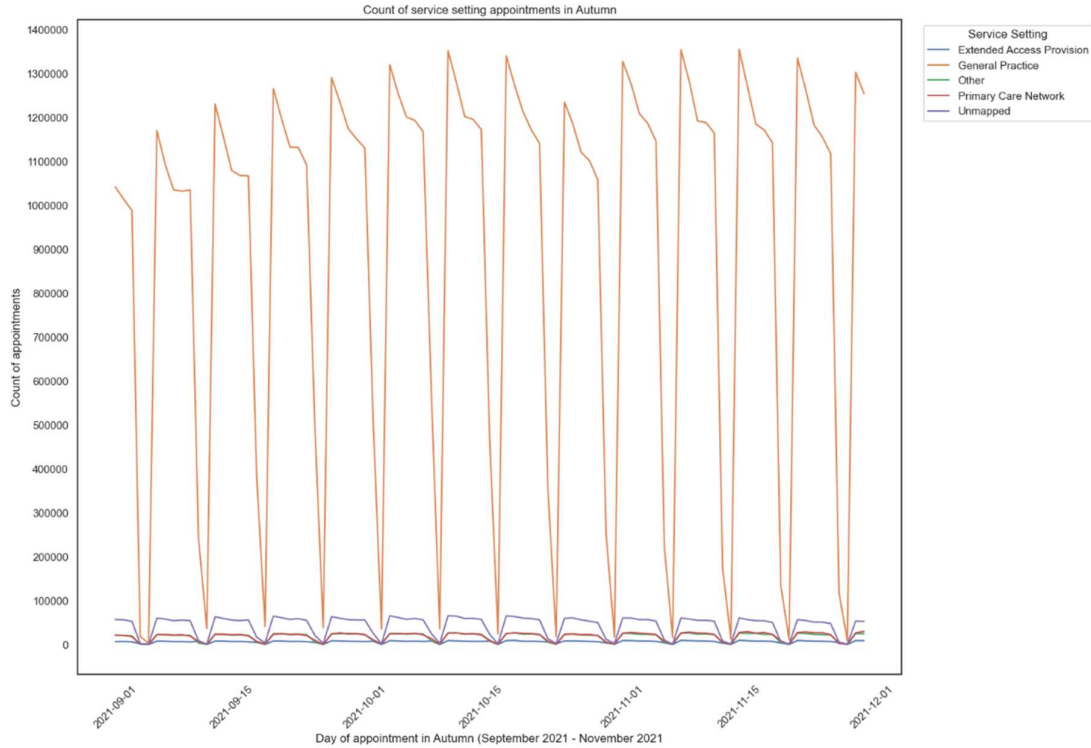


Figure 12 – A seasonal lineplot

Twitter data was read in Python, and a list of the hashtags was extracted using a for loop into a DataFrame, and then visualised with a horizontal barplot. Two hashtags were more popular than others: “#healthcare’ and ‘#health.’ When the two most popular hashtags were removed, the distribution of other hashtags was much clearer. However, it indicated that the other hashtags were more varied and not necessarily linked with

health – “#ai” and “#marketing” (see figure 13).

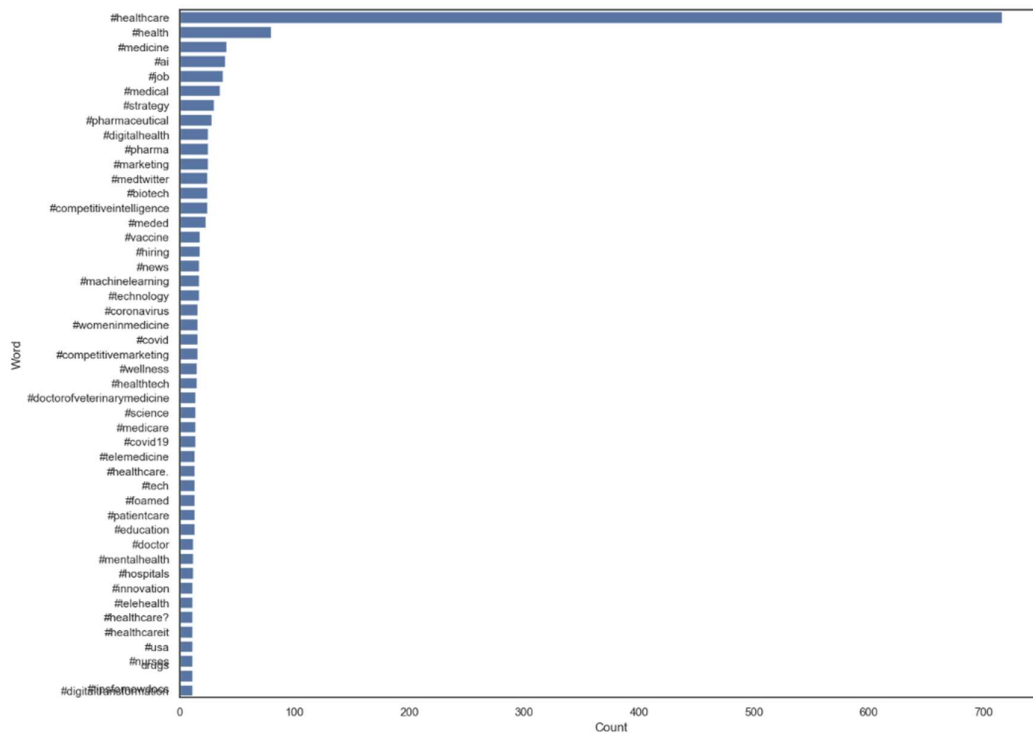


Figure 13 – a bar chart showing #healthcare and #health

A dataset was filtered to look at specific dates (see figure 14). The filtered DataFrame had a number of different aspects, but the main focus was the sum of the count of appointments. A Series was created for utilisation which comprised sum of count of appointments divided by 30. Visualisations of the appointment count and the average utilisation demonstrated that the NHS GP surgeries were under considerable strain during the autumn of 2021, with a peak in November 2021 (see figure 15).

```
# Filter the data set to only look at data from 2021-08 onwards.  
ar = ar[ar['appointment_month'] >= '2021-08']
```

Figure 14 – Python code showing the filter for specific dates.

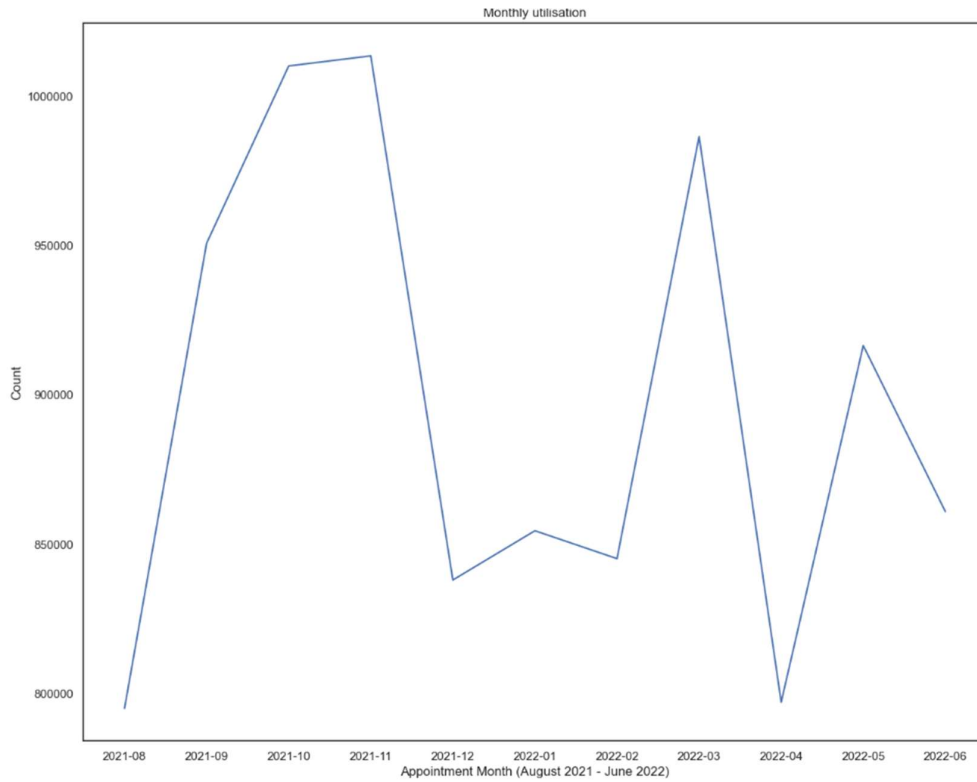


Figure 15 – Time series of utilisation average per month

When healthcare professionals were investigated it illustrated that there were extra staff required during the peak in October/November 2021. This is evident by the amount of “Other Practice Staff” taking appointments in October, and both being high in November (see figure 16).

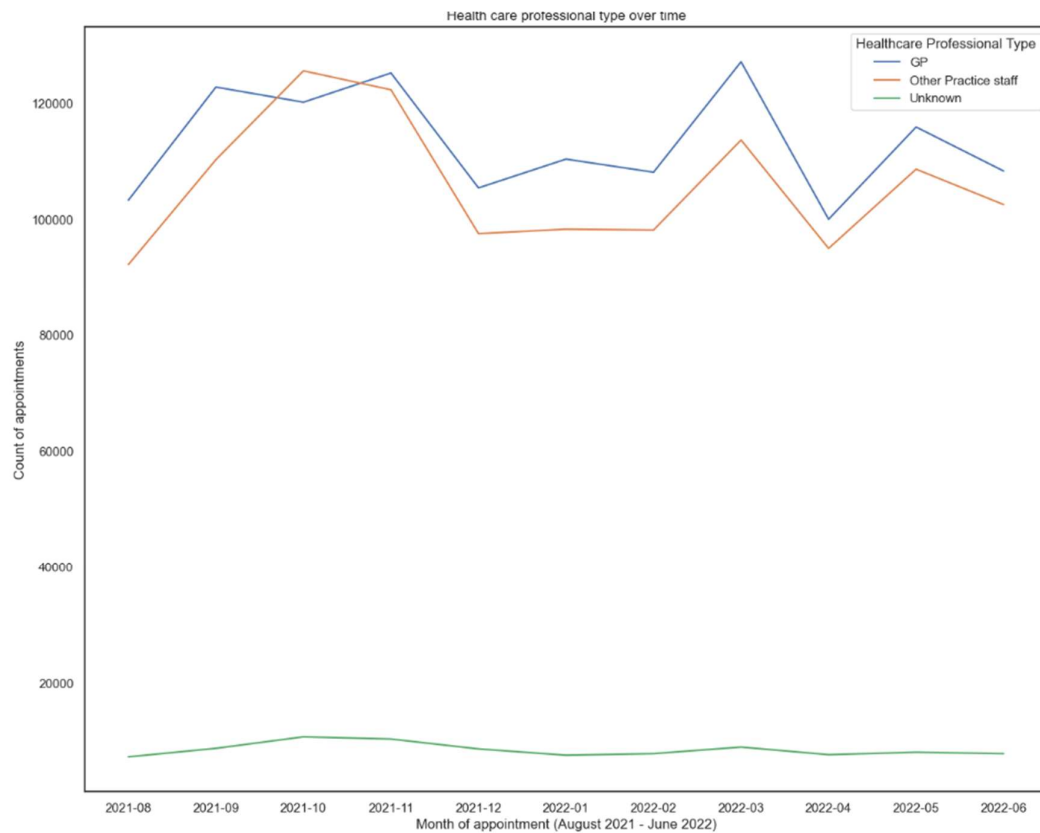


Figure 16 – A time series to show the distribution of health care professionals over time

Appointment status investigations demonstrated that the vast majority of appointments were attended, but this warranted further investigation into any of correlation towards

DNAs (see figure 17).

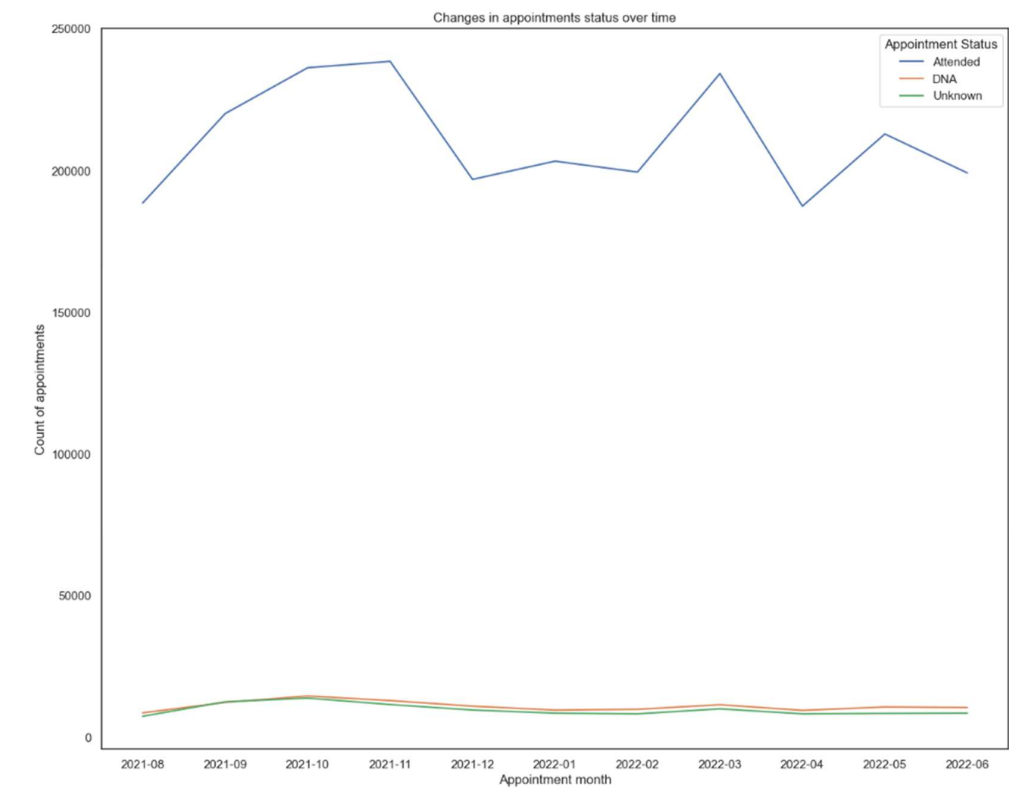


Figure 17 – A time series of the distribution of appointment statuses.

The time series for time between booking and the appointment categories identified that most of the appointments were booked same-day. This is evidence of the 8AM scramble which was fixed under a new contract agreed in 2025 (BBC, 2025) (see figure

18).

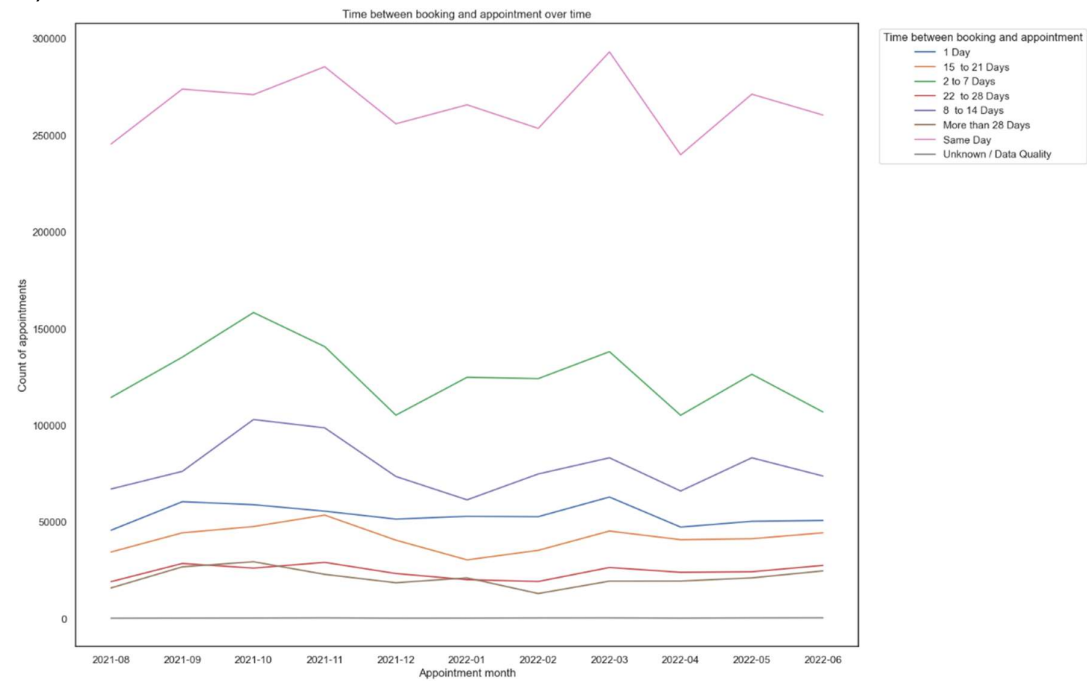


Figure 18 – A time series comparing different booking lengths for appointments

When appointment mode was investigated it identified face-to-face appointment were the most popular and it followed a similar path to the utilisation lineplot previously. This suggests that a lot of a GPs time was taken up by face-to-face appointments (see figure

19).

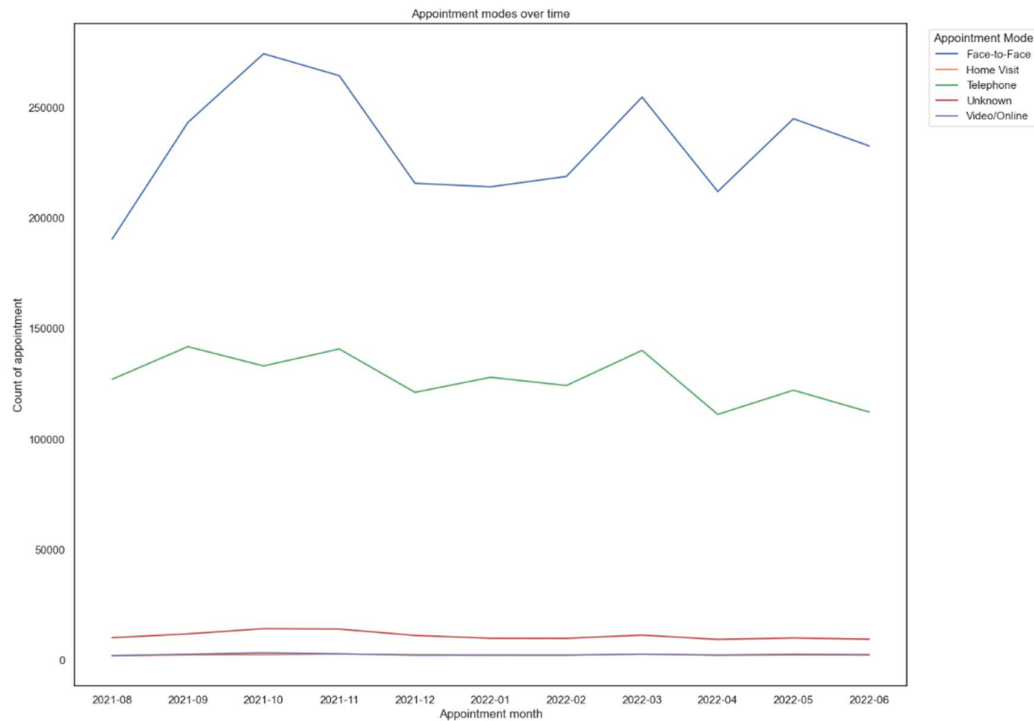


Figure 19 – A time series comparing different appointment modes

A boxplot was created focusing on the service setting and count of appointments indicated that the general practice appointments were much larger than the rest of the options (see figure 20). Omitting general practice allowed to see a more even distribution of the other categories. It showed that the unmapped category the second largest distribution (see figure 21).

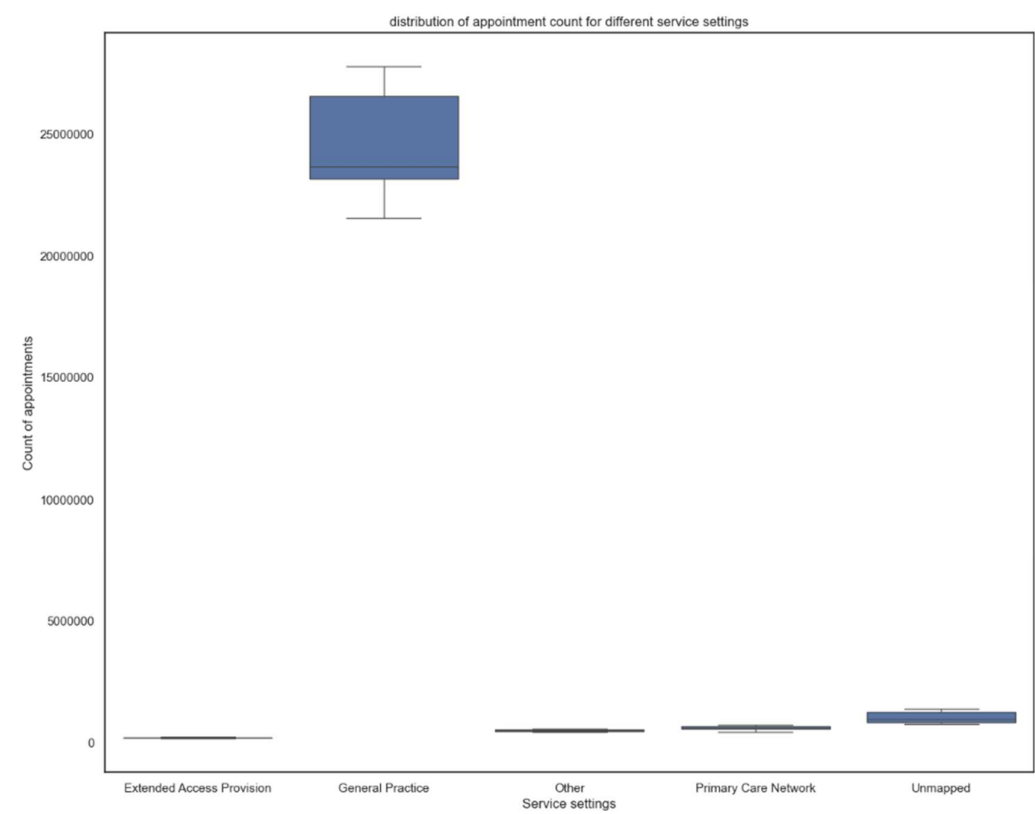


Figure 20 – a boxplot showing different distributions for service settings. General practice makes the rest difficult to see

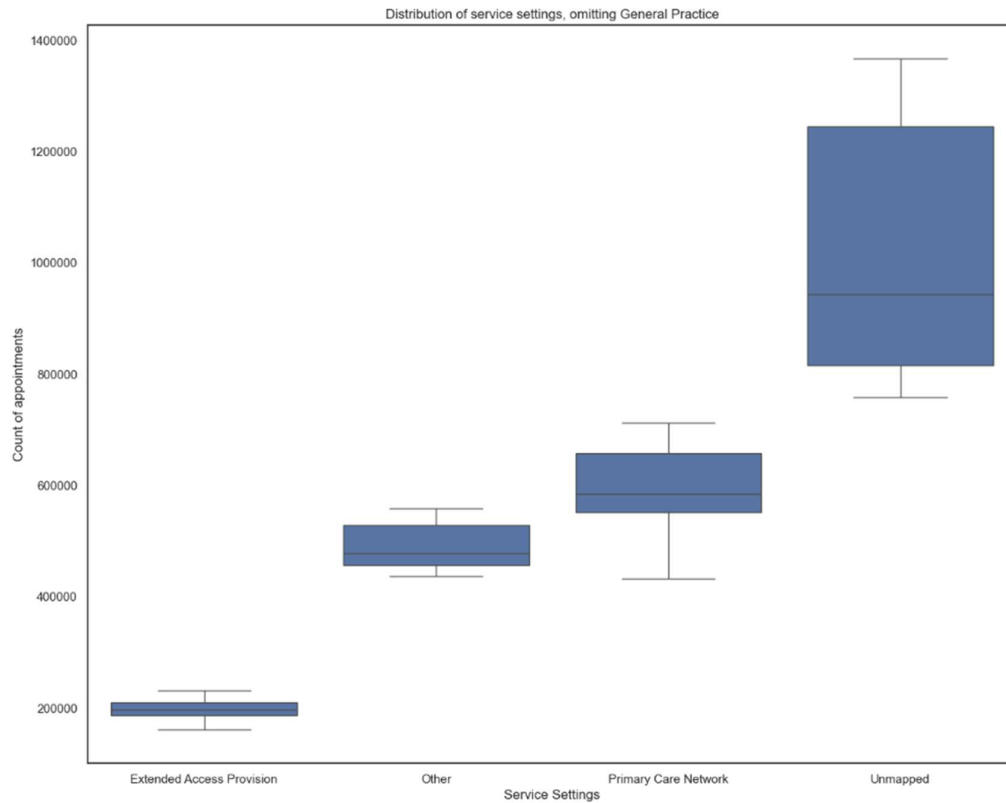


Figure 21 – a boxplot omitting General practice that shows unmapped having a wide distribution.

A grouped DataFrame to look at the relationship between time between booking and the appointment and count of appointment was filtered to look at 'DNA.' This was then visualised and it revealed that most appointments are labelled as DNA if they are booked between 2 – 7 days prior to the appointment. The same-day category highlighted a lot of DNAs also (see figure 22). When the hue of the barplot on appointment mode was used, it revealed that all the categories demonstrated that the face-to-face was much larger relative to the other categories. However, for same-day, the telephone option was larger compared to the others. This suggests that the same-day telephone appointment is less successful because of the window that is given for patients.

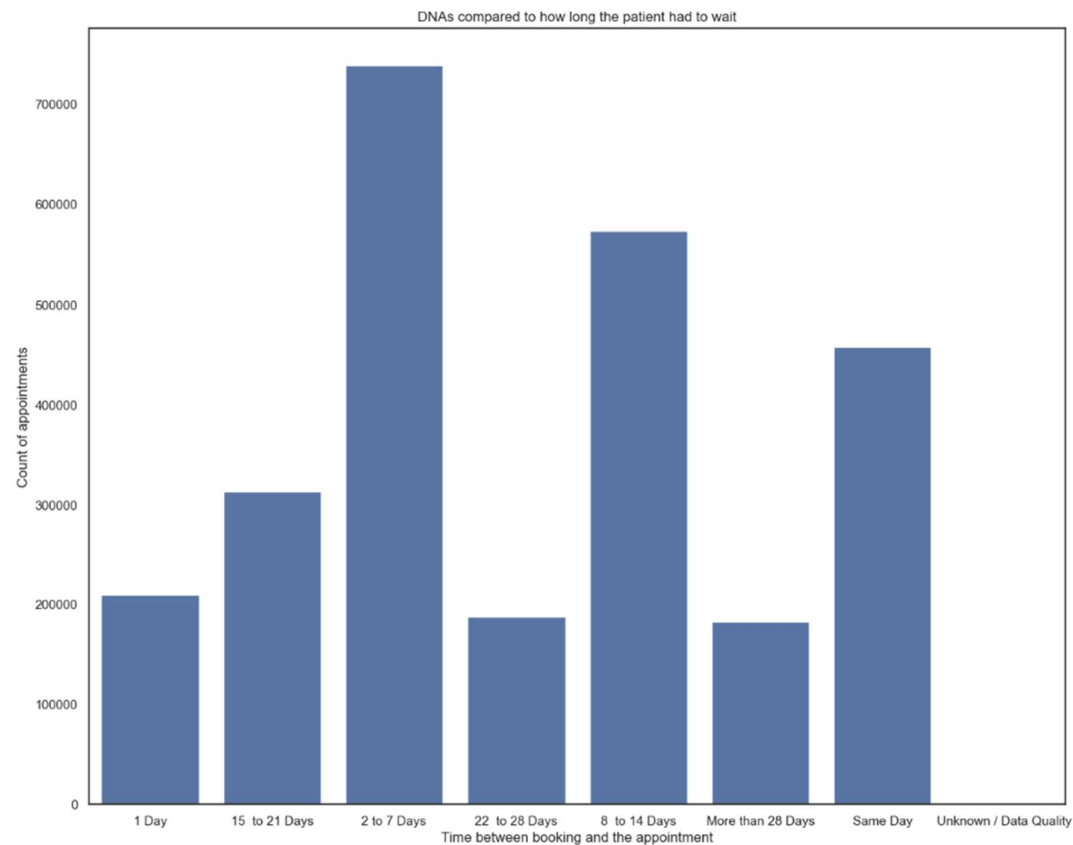


Figure 22 – A barplot for DNA compared to booking wait time

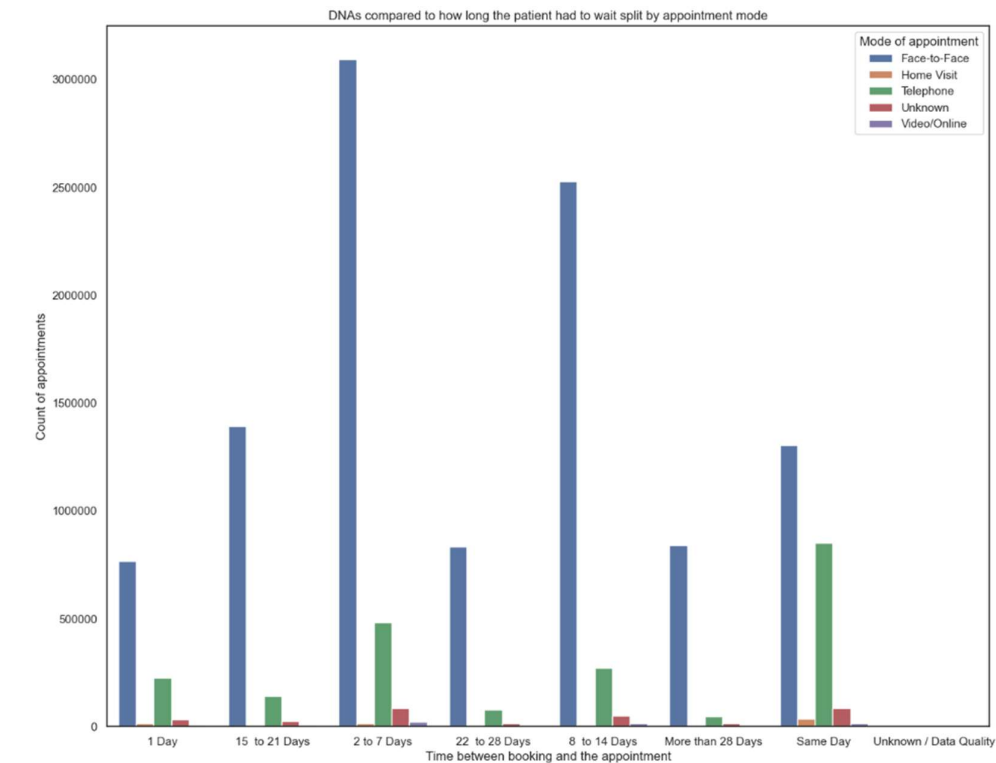


Figure 23 – A barplot for DNAs compared to booking wait times broken down by mode of appointment

Recommendations

To conclude; when looking at the data that was available there was a pattern. It shows at times that the NHS across England was near to full capacity, which is understandable considering the context of the country dealing with the aftereffects of the covid lockdowns. I recommend that the NHS increases the capacity to deal with increases in the population in the future. There is room in the system for the surgeries to be used at weekends and on bank holidays to increase appointments. However, there are also efficiencies that could be fixed. Allowing GPs to do more telephone appointments rather than face-to-face will help with streamlining, but this can only be achieved if the patient is given an exact time rather than a window for a call, to avoid any missed appointments that are booked on the same-day. Similarly, for other appointments that are booked a fortnight in advance, reminder messages can be sent out so that the patient has a better opportunity to be available for an appointment. Ultimately, it shows that a two-pronged approach towards the NHS capacity will increase the efficiency of the primary care system. By keeping the patients more up to date and having more specific times to avoid missed appointments, as well as increasing staffing levels and more appointments during weekends and bank holidays, there will be more room in this system to be used.

Appendix

Figure 1

```
# Import the necessary libraries.
import pandas as pd
import numpy as np

# Optional - Ignore warnings.
import warnings
warnings.filterwarnings('ignore')
```

Figure 1 - Python code showing importing Pandas and Numpy libraries.

Figure 2

```
# Determine the number of Locations.
print(nc['sub_icb_location_name'].value_counts().count())
print(ad['sub_icb_location_ons_code'].value_counts().count())
print("There are",
      nc['sub_icb_location_name'].value_counts().count(),
      "different sub-icb locations.")

106
106
There are 106 different sub-icb locations.

# Determine the number of wider Locations.
print(nc['icb_ons_code'].value_counts().count())
print(f"There are {nc['icb_ons_code'].value_counts().count()} wider locations, however we do not have their names, only their ONS location codes.")

42
There are 42 wider locations, however we do not have their names, only their ONS location codes.
```

Figure 2 - Python code for the number of locations

Figure 3

```
# Determine the top five locations based on record count.
nc_record_count = nc['sub_icb_location_name'].value_counts()

print(f"The top 5 locations based on record count:
      {nc_record_count.head(5).to_string(header=False)}")
```

The top 5 locations based on record count:

NHS North West London ICB - W2U3Z	13007
NHS Kent and Medway ICB - 91Q	12637
NHS Devon ICB - 15N	12526
NHS Hampshire and Isle Of Wight ICB - D9Y0V	12171
NHS North East London ICB - A3A8R	11837

Figure 3 – Python Code to determine the top five locations

Figure 4

```
# Determine the number of service settings.
print(nc['service_setting'].value_counts())
print(f"""There are {nc['service_setting'].value_counts().count()} different service settings.""")

service_setting
General Practice          359274
Primary Care Network      183790
Other                    138789
Extended Access Provision 108122
Unmapped                  27419
Name: count, dtype: int64
There are 5 different service settings.

# Determine the number of context types.
print(nc['context_type'].value_counts())
print(f"""There are {nc['context_type'].value_counts().count()} different context types.""")

context_type
Care Related Encounter    700481
Inconsistent Mapping      89494
Unmapped                  27419
Name: count, dtype: int64
There are 3 different context types.
```

Figure 4 – Some examples of Python code to understand different aspects of the data sets.

Figure 5

```
# Determine the minimum and maximum dates in the nc DataFrame.
# Use appropriate docstrings.
print(nc['appointment_date'].max().date())
print(nc['appointment_date'].min().date())
print(f"""The National Category data set starts at {nc['appointment_date'].min().date()}, and ends at {nc['appointment_date'].max().date()}.""")

2022-06-30
2021-08-01
The National Category data set starts at 2021-08-01, and ends at 2022-06-30.
```

Figure 5 – An example of some Python code to check the start and end dates of the data sets.

Figure 6

```
# Filter the dates of the appointments.
nc_NorthWestLondon = nc_NorthWestLondon[
    (nc_NorthWestLondon['appointment_date'] >= '2022-01-01')
    &
    (nc_NorthWestLondon['appointment_date'] <= '2022-06-01')]

# Sense check the filtered DataFrame.
print(nc_NorthWestLondon.shape)
nc_NorthWestLondon
```

Figure 6 – Showing some Python code that filters the DataFrame.

Figure 7

```
# Determine the total number of appointments per month grouped by year and month.
nc['count_of_appointments'].groupby([
    nc['appointment_date'].dt.year,
    nc['appointment_date'].dt.month
]).sum().sort_values(ascending=False)
```

appointment_date	appointment_date	
2021	11	30405070
	10	30303834
2022	3	29595038
2021	9	28522501
2022	5	27495508
	6	25828078
	1	25635474
	2	25355260
2021	12	25140776
2022	4	23913060
2021	8	23852171

Figure 7 – Python code using the groupby method to see the busiest month.

Figure 8

```
# Import the necessary Libraries.
import seaborn as sns
import matplotlib.pyplot as plt

# Set figure size.
sns.set(rc={'figure.figsize':(15, 12)})

# Set the plot style as white.
sns.set_style('white')
```

Figure 8 – Python code preparing the workstation for visualisations.

Figure 9

```
# Aggregate on monthly level and determine the sum of records per month.
nc_ss = nc.groupby(['appointment_month', 'service_setting'])\
['count_of_appointments'].sum().reset_index()

# View output.
nc_ss.head()
```

Figure 9 – Python code showing subsets using the groupby method.

Figure 10

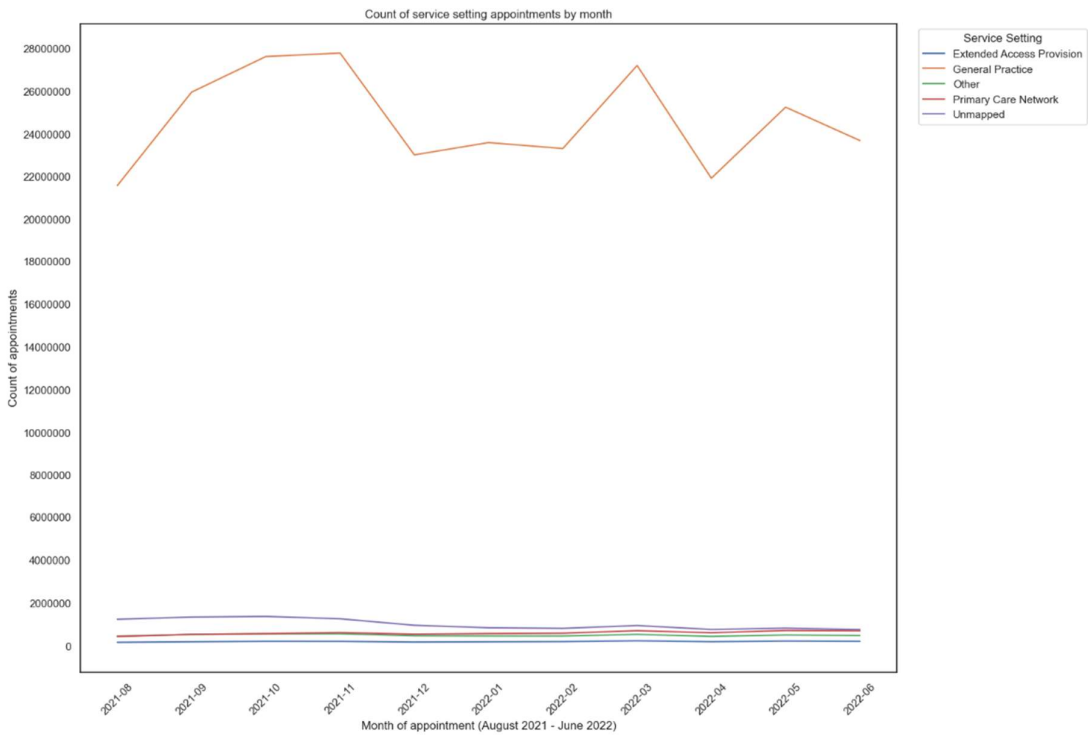


Figure 10 – Time series of service setting showing more General practice.

Figure 11

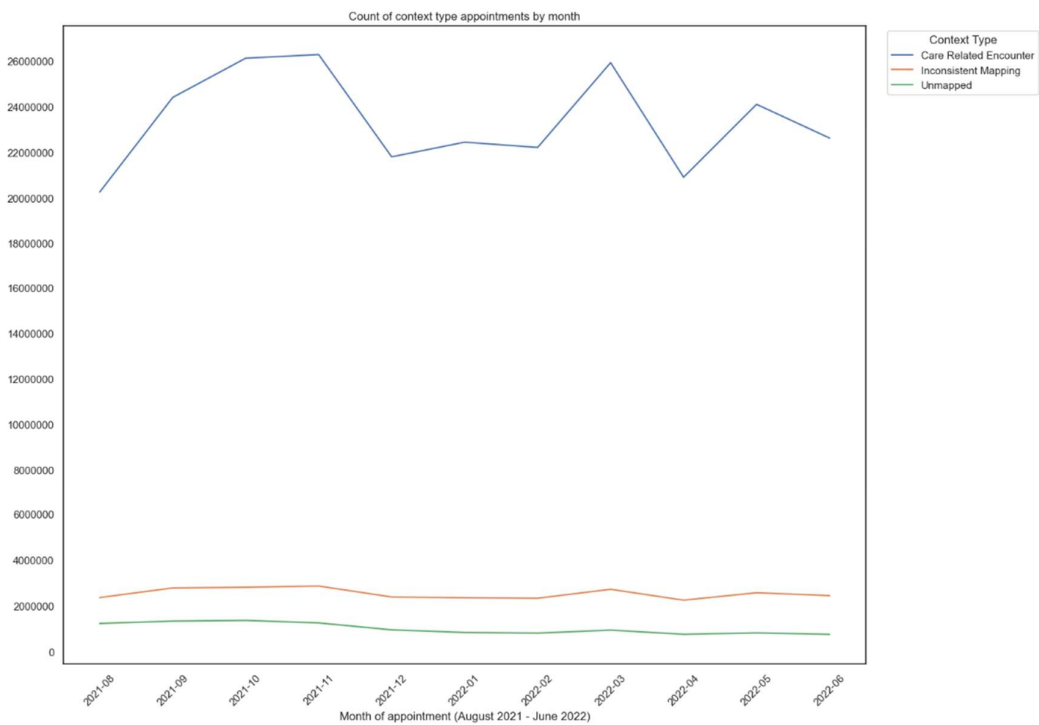


Figure 11 – a time series to show different context types over a year. Care related encounter is the most observed

Figure 12

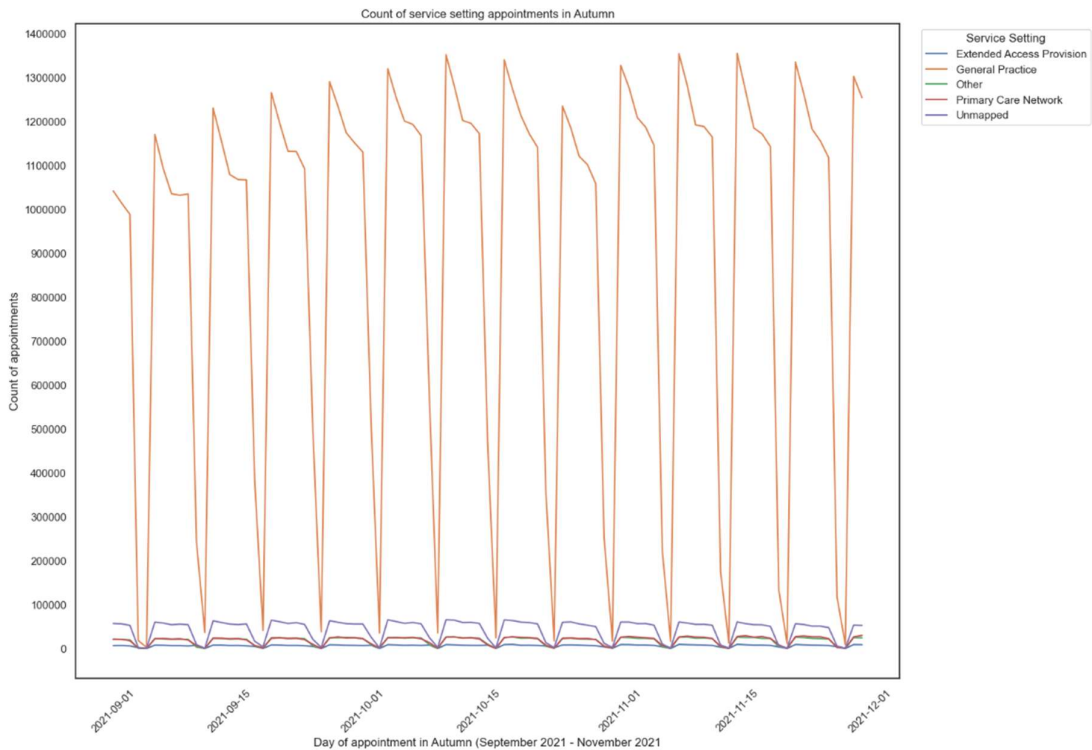


Figure 12 - A seasonal lineplot

Figure 13

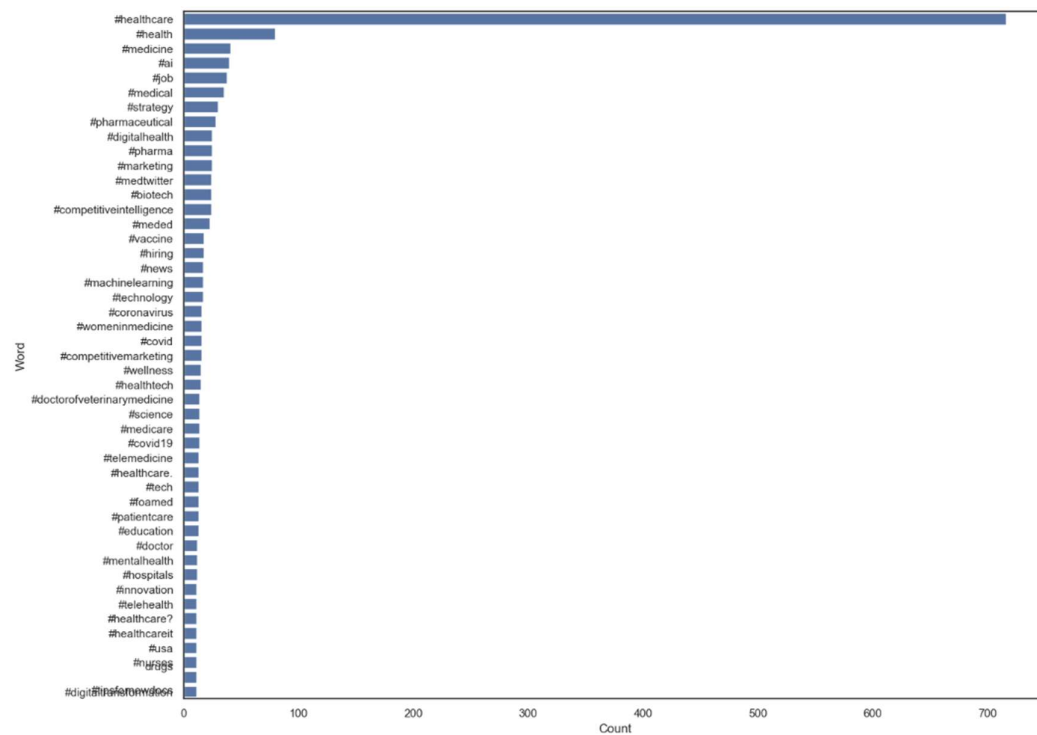


Figure 13 - A bar plot showing #healthcare and #health

Figure 14

```
# Filter the data set to only look at data from 2021-08 onwards.  
ar = ar[ar['appointment_month'] >= '2021-08']
```

Figure 14 – Python code showing the filter for specific dates.

Figure 15

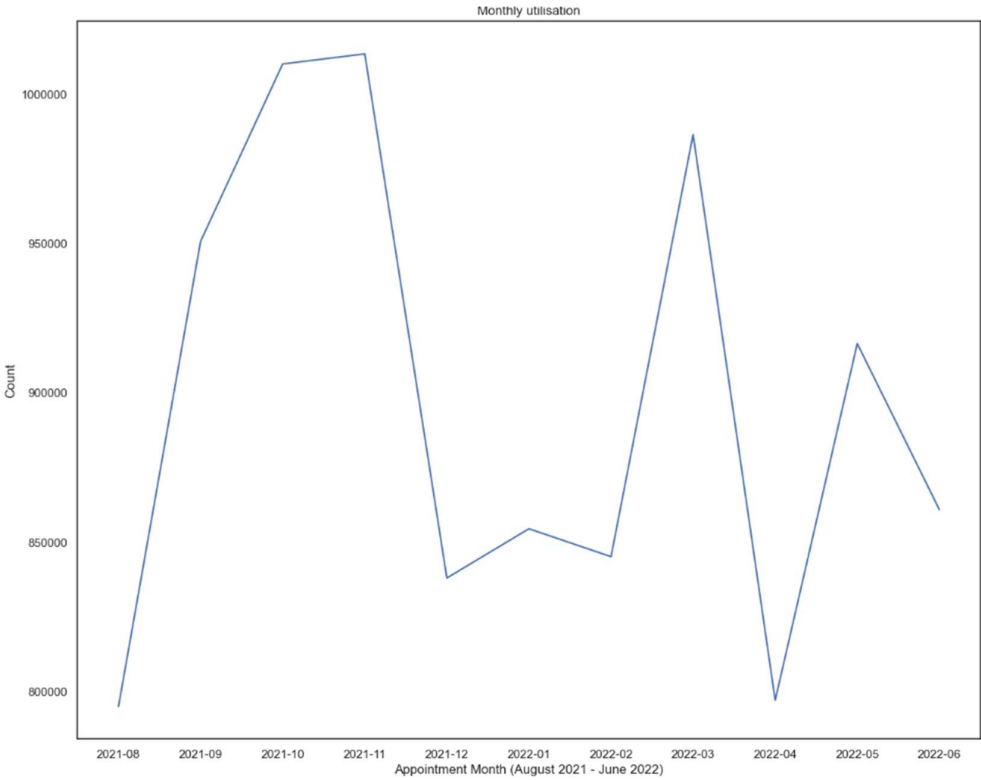


Figure 15 – Time series of utilisation average per month

Figure 16

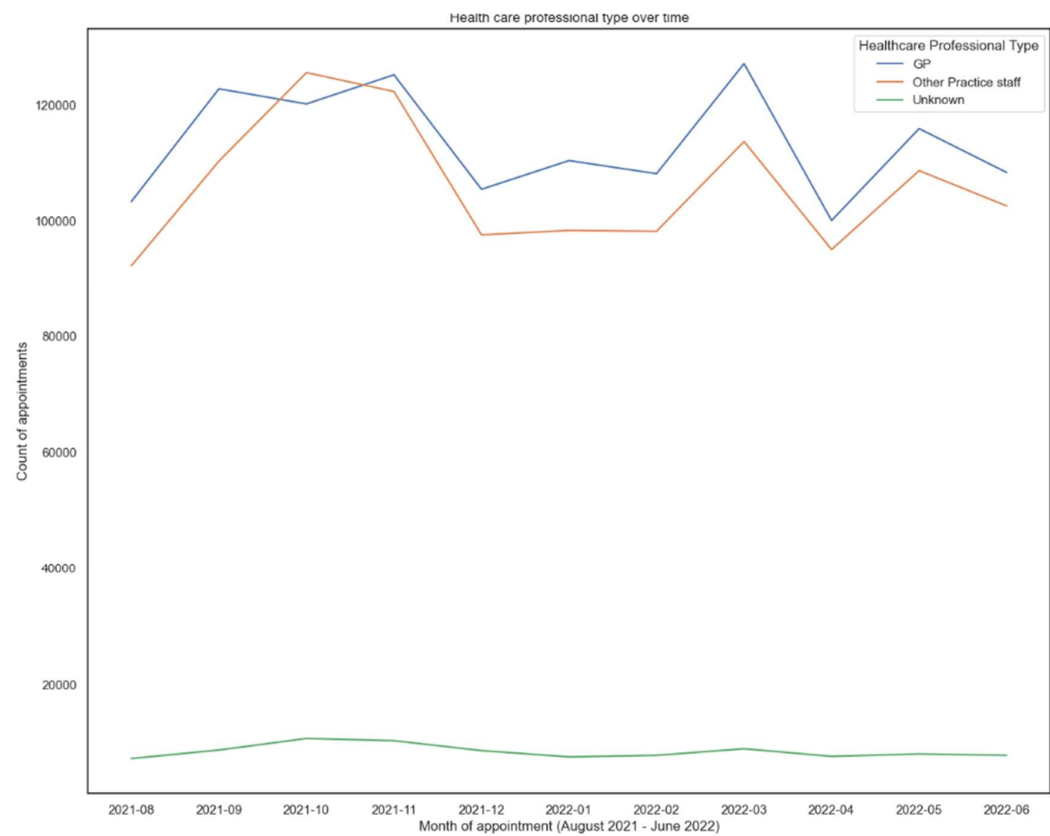


Figure 16 – A time series to show the distribution of health care professionals over time

Figure 17

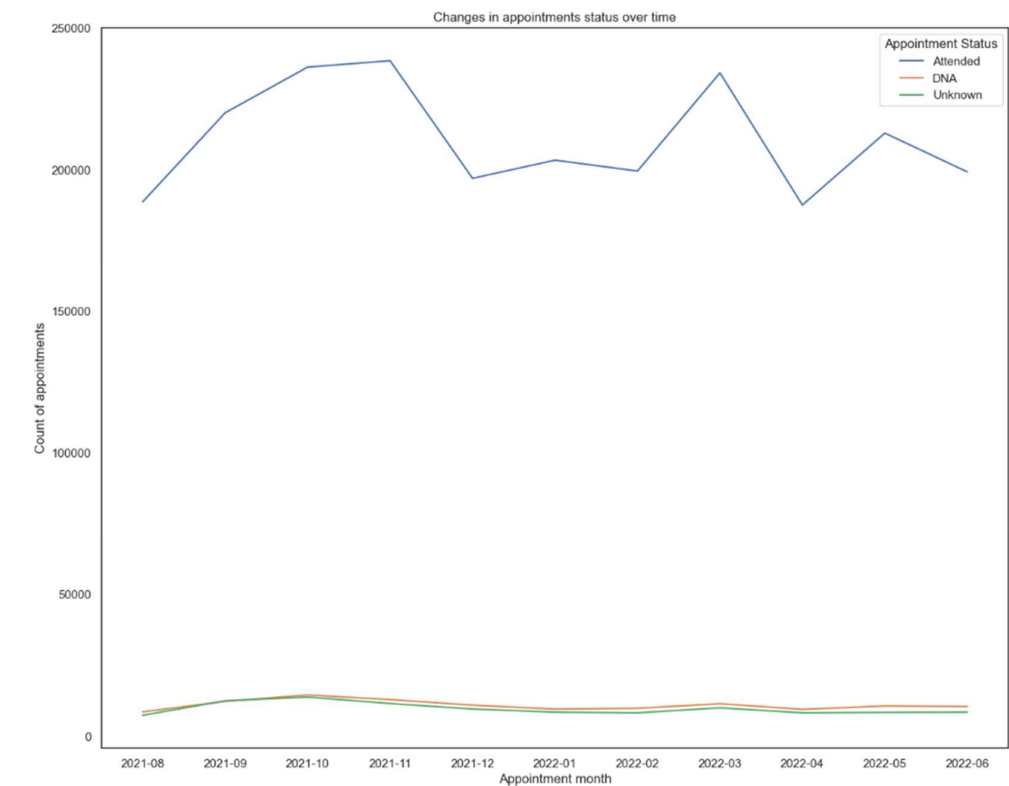


Figure 17 – A time series of the distribution of appointment statuses.

Figure 18

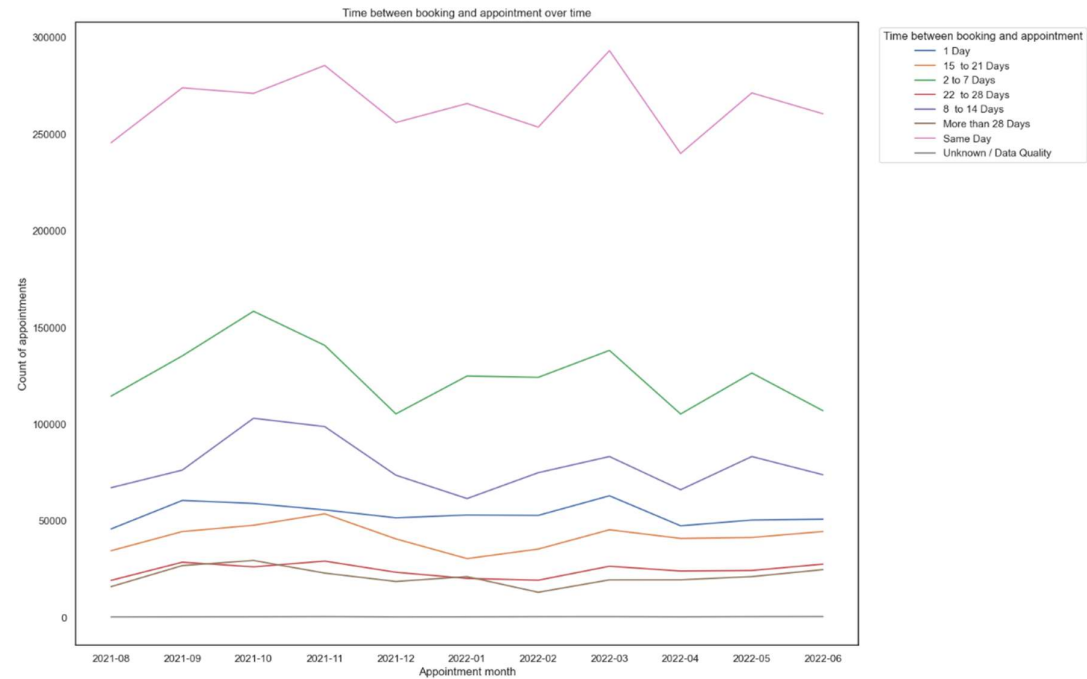


Figure 18 – A time series comparing different booking lengths for appointments

Figure 19

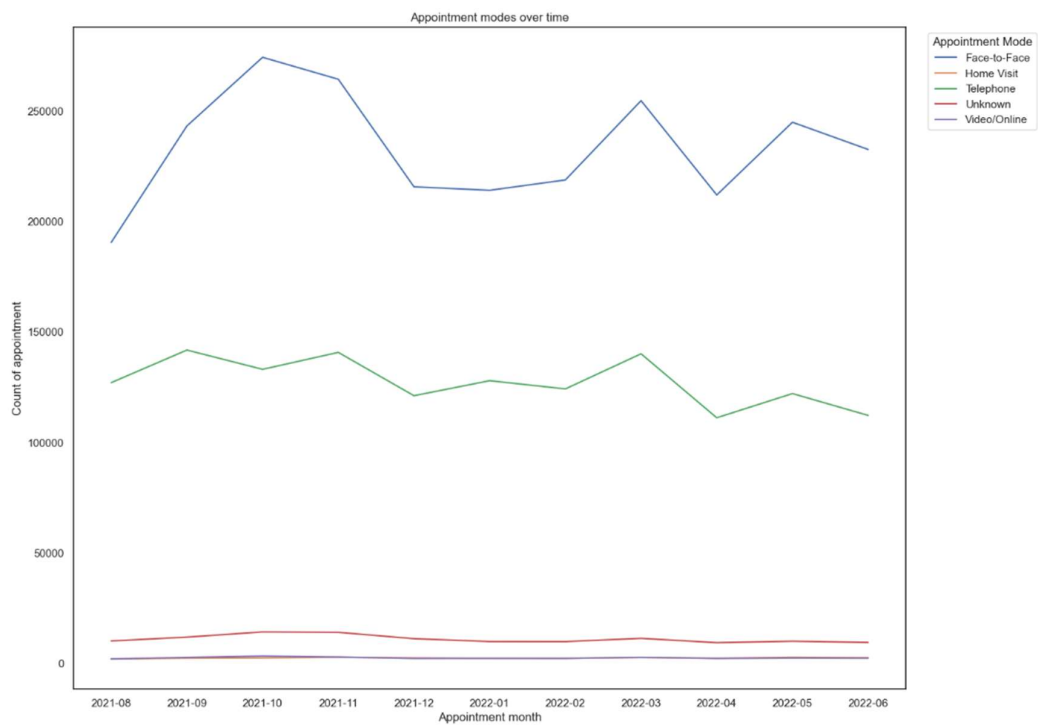


Figure 19 – A time series comparing different appointment modes

Figure 20

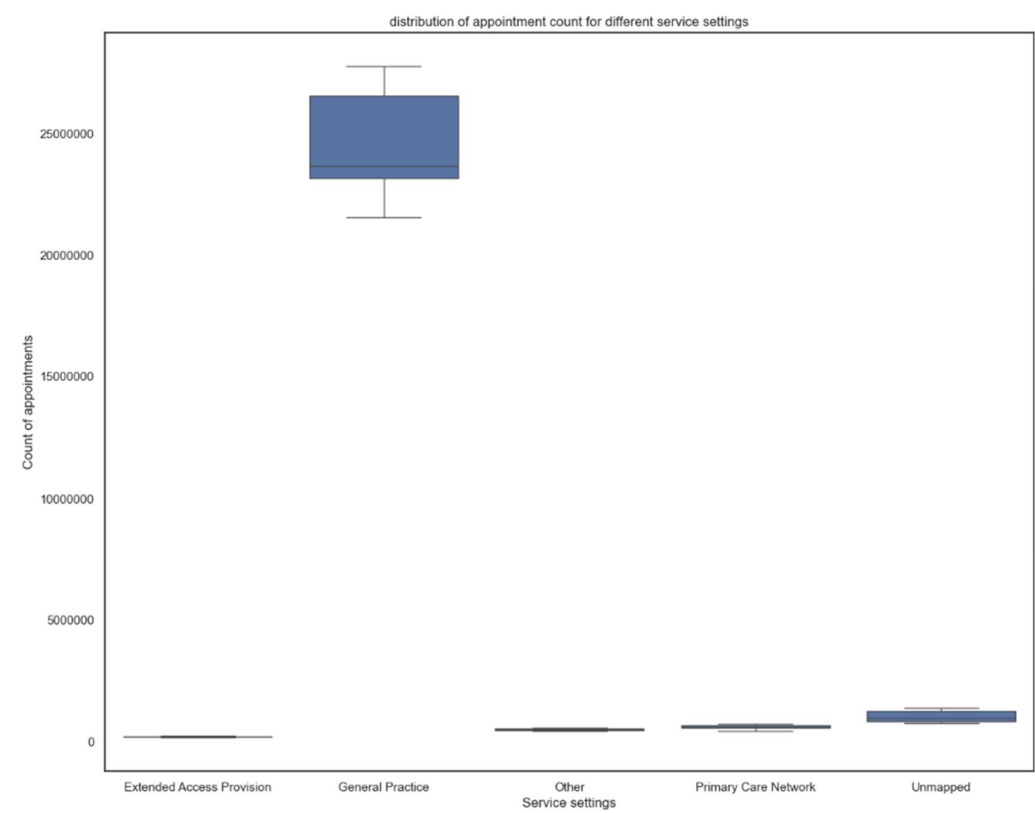


Figure 20 – a boxplot showing different distributions for service settings. General practice makes the rest difficult to see

Figure 21

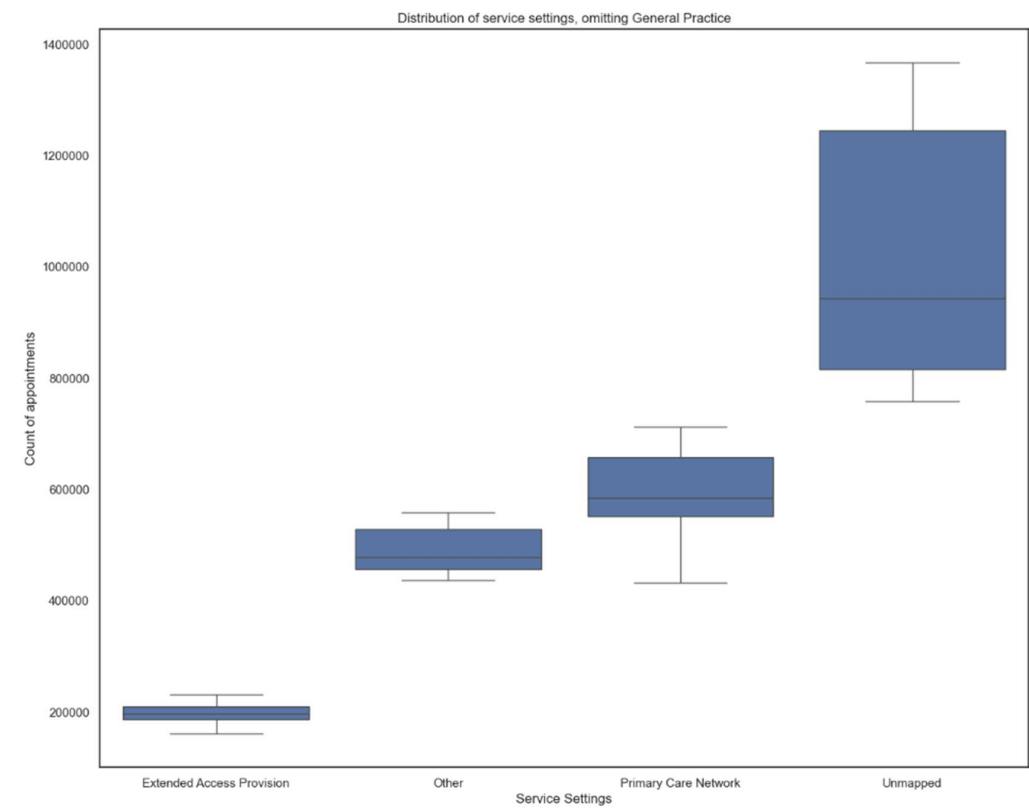


Figure 21 – a boxplot omitting General practice that shows unmapped having a wide distribution.

Figure 22

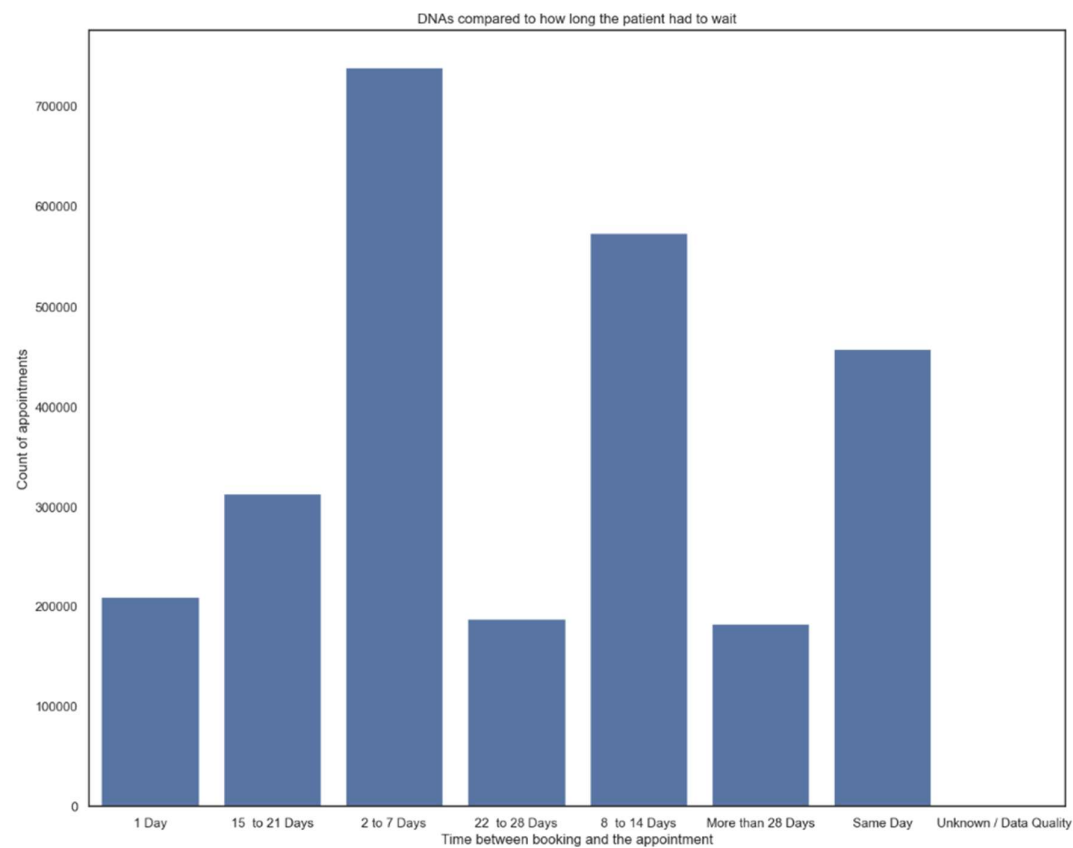


Figure 22 – A barplot for DNA compared to booking wait time

Figure 23

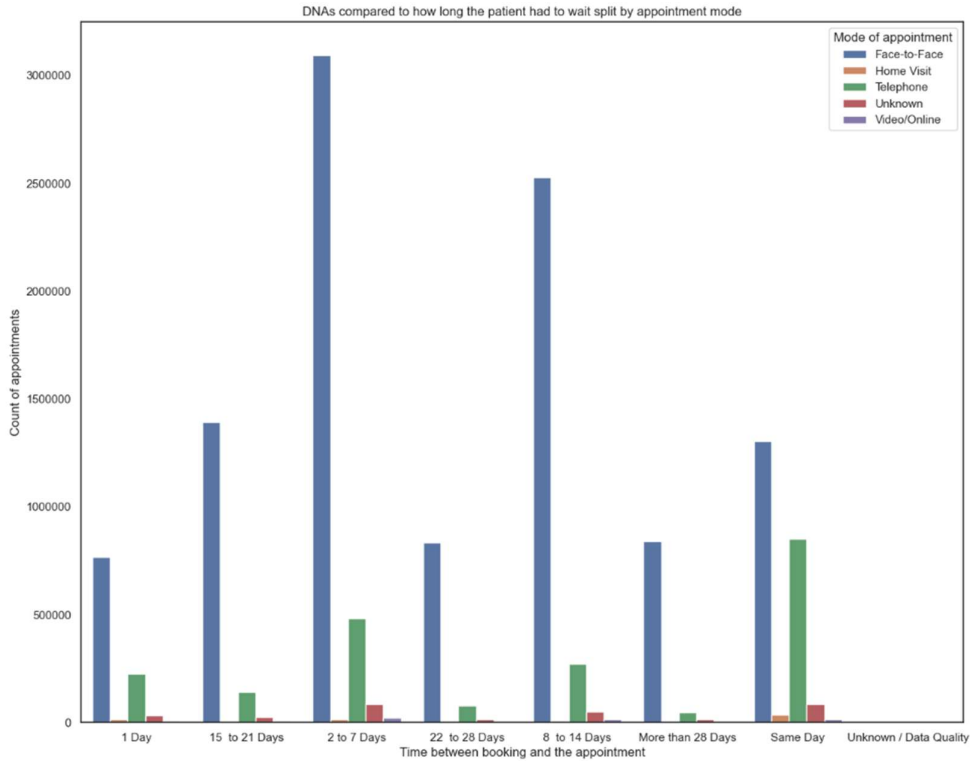


Figure 23 – A barplot for DNAs compared to booking wait times broken down by mode of appointment

References

BBC (2025) *GP appointment '8am scramble' to be overhauled under new NHS contract*. Available at: <https://www.bbc.co.uk/news> (Accessed: 25 May 2026).

NHS Digital (2021) *Appointments in General Practice, November 2021*. Available at: <https://digital.nhs.uk/data-and-information/publications/statistical/appointments-in-general-practice/november-2021> (Accessed: 25 May 2026).